



ft\_irc

## インターネット・リレー・チャット

**概要:** このプロジェクトでは、独自のIRCサーバーを作成します。

実際のIRCクライアントを使用して、自分のサーバーに接続し、テストを行います。

インターネットは、接続されたコンピュータ同士が相互に通信できるようにする、確固たる標準プロトコルによって支えられています。

これらのプロトコルを理解しておくことは常に有益です。

バージョン : 10.0

# 目次

|            |                  |           |
|------------|------------------|-----------|
| <b>I</b>   | <b>はじめに</b>      | <b>2</b>  |
| <b>II</b>  | <b>一般的なルール</b>   | <b>3</b>  |
| <b>III</b> | <b>AIへの指示</b>    | <b>4</b>  |
| <b>IV</b>  | <b>必須項目</b>      | <b>6</b>  |
| IV.1       | 要件 .....         | 8         |
| IV.2       | MacOSのみ .....    | 9         |
| IV.3       | テスト例 .....       | 9         |
| <b>V</b>   | <b>Readme 要件</b> | <b>10</b> |
| <b>VI</b>  | <b>ボーナスパート</b>   | <b>11</b> |
| <b>VII</b> | <b>提出と相互評価</b>   | <b>12</b> |



# 第I章 はじめに

インターネット・リレー・チャット（IRC）は、インターネット上のテキストベースの通信プロトコルです。公開または非公開のリアルタイムメッセージング機能を提供します。ユーザーはダイレクトメッセージを交換したり、グループチャンネルに参加したりすることができます。

IRCクライアントは、チャンネルに参加するためにIRCサーバーに接続します。IRCサーバーは相互に接続され、ネットワークを形成しています。

## 第II章 一般的なルー

## ル

- プログラムは、いかなる状況下でも（メモリ不足の場合であっても）クラッシュしてはならず、予期せず終了してはなりません。  
万が一、そのような事態が発生した場合、そのプロジェクトは機能していないものとみなされ、評価は0となります。
- ソースファイルをコンパイルするための Makefile を提出する必要があります。不必要な再リンクを行ってはなりません。
- Makefile には、少なくとも以下のルールを含める必要があります。  
\$(NAME)、all、clean、fclean、および re。
- c++ を使用して、-Wall、-Wextra、-Werror のフラグを指定してコードをコンパイルしてください。
- コードはC++98規格に準拠している必要があります。そのため、-std=c++98フラグを追加しても、依然としてコンパイルが成功するはずです。
- 利用可能な場合は常にC++の機能を使用してコーディングするようにしてください（例えば、<string.h>の代わりに<cstring>を選択するなど）。C関数の使用は許可されていますが、可能であれば常にC++版を優先してください。
- 外部ライブラリおよびBoostライブラリの使用は一切禁止されています。

## 第III章

# AIに関する指示

### ● 背景

学習の過程において、AIはさまざまなタスクを支援してくれます。時間をかけて、AIツールの多様な機能や、それらがどのようにあなたの作業をサポートできるかを探求してください。ただし、常に慎重に扱い、結果を批判的に評価するようにしてください。コード、ドキュメント、アイデア、技術的な説明のいずれであっても、あなたの質問が適切に構成されていたか、生成されたコンテンツが正確であることを完全に確信することはできません。仲間は、間違いや見落としを防ぐための貴重なリソースです。

### ● 主なメッセージ

- ➡ AIを活用して、反復的または面倒な作業を軽減しましょう。
- ➡ 将来のキャリアに役立つ、コーディングおよび非コーディングの両面におけるプロンプティングスキルを磨きましょう。
- ➡ AIシステムの仕組みを理解し、よくあるリスク、バイアス、倫理的な問題をよりの確に予測・回避するようにしましょう。
- ➡ 仲間と協力し、技術スキルとリーダーシップスキルの両方を磨き続けましょう。
- ➡ 自分が完全に理解し、責任を負えるAI生成コンテンツのみを使用しましょう。

### ● 学習者のルール：

- AIツールをじっくりと探求し、その仕組みを理解する時間を確保してください。そうすることで、倫理的にツールを利用し、潜在的なバイアスを軽減することができます。
- プロンプトを入力する前に、課題について熟考してください。これにより、正確な語彙を用いて、より明確で詳細かつ適切なプロンプトを作成できるようになります。
- AIによって生成されたものはすべて、体系的に確認、見直し、疑問を持ち、検証する習慣を身につけてください。
- 常にピアレビューを求めるべきです。自分自身の検証だけに頼ってはいけません。 4

## ● フェーズの成果：

- 汎用的なプロンプティングスキルと、特定の分野に特化したプロンプティングスキルの両方を身につける。
- AIツールを効果的に活用して生産性を向上させる。
- コンピュータショナル・シンキング、問題解決能力、適応力、および協働能力を引き続き強化する。

## ● コメントと例：

- 試験や評価など、真の理解を示さなければならない状況に頻繁に直面することになります。準備を整え、技術的スキルと対人スキルの両方を磨き続けましょう。
- 自分の考え方を説明したり、仲間と議論したりすることで、理解の不足が明らかになることがよくあります。仲間との学びを優先しましょう。
- AIツールは、多くの場合、あなたの具体的な文脈を把握しておらず、一般的な回答を提供しがちです。同じ環境を共有する仲間なら、より適切で正確な洞察を提供してくれるでしょう。
- AIが最も可能性の高い答えを生成する傾向にあるのに対し、仲間は別の視点や貴重なニュアンスを提供してくれます。質を確かめるチェックポイントとして、仲間を頼りにしましょう。

### ✓ 良い実践例：

AIに「ソート関数をテストするにはどうすればいいですか？」と尋ねます。AIはいくつかのアイデアを提示してくれます。それらを試してみて、結果を仲間と確認します。そして、二人でアプローチを洗練させていきます。

### ✗ 悪い実践例：

AIに関数全体を作成させ、それを自分のプロジェクトにコピー＆ペーストする。ピア評価の際、その関数が何をするのか、なぜそうするのかを説明できない。信頼を失い、プロジェクトに失敗してしまう。

### ✓ 良い実践例：

AIを活用してパーサーの設計を支援します。その後、仲間と一緒にロジックを検証します。2つのバグを見つけ、協力して書き直します。その結果、より良く、よりクリーンで、完全に理解されたコードが完成します。

### ✗ 悪い実践例：

プロジェクトの重要な部分のコードをCopilotに生成させた。コンパイルは通ったが、パイプの処理方法を説明できなかった。評価の際、その正当性を説明できず、プロジェクトに失敗した。

## 第IV章 必須課題

|          |                                                                                                                                                                                                                                                                                                                                      |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| プログラム名   | ircserv                                                                                                                                                                                                                                                                                                                              |
| 提出ファイル   | Makefile、*.h、*.hpp、*.cpp、*.tpp、*.ipp、<br>オプションの設定ファイル                                                                                                                                                                                                                                                                                |
| Makefile | NAME, all, clean, fclean, re                                                                                                                                                                                                                                                                                                         |
| 引数       | ポート: リスニングポート<br>password: 接続パスワード                                                                                                                                                                                                                                                                                                   |
| 外部関数     | C++ 98 のすべての機能。<br>socket、close、setsockopt、getsockname、getprotobyname、<br>gethostbyname、getaddrinfo、freeaddrinfo、bind、connect、<br>listen、accept、htons、htonl、ntohs、ntohl、inet_addr、<br>inet_ntoa、inet_ntop、send、recv、signal、sigaction、<br>sigemptyset、sigfillset、sigaddset、sigdelset、sigismember<br>、lseek、fstat、fcntl、poll（または同等の関数） |
| Libft 認定 | 該当なし                                                                                                                                                                                                                                                                                                                                 |
| 説明       | C++ 98 による IRC サーバー                                                                                                                                                                                                                                                                                                                  |

C++ 98 標準を使用して IRC サーバーを開発する必要があります。

IRCクライアントを開発しては**なりません**。

サーバー間通信を実装しては**なりません**。

作成した実行ファイルは、次のように実行されます。

```
./ircserv <port> <password>
```

- port: IRC サーバーが着信する IRC 接続を待機するポート番号。
- password: 接続用のパスワード。サーバーへの接続を試みるすべてのIRCクライアントで必要となります。



題名や評価基準には `poll()` が記載されていますが、`select()`、`kqueue()`、`epoll()` などの同等の関数を任意に使用しても構いません。

## IV.1 要件

- サーバーは、ハングすることなく複数のクライアントを同時に処理できる必要があります。
- フォークは禁止されている。すべてのI/O操作は**ノンブロッキング**でなければならない。
- これらすべての操作（読み取り、書き込み、さらにリスンなど）を処理するために使用できるのは、`poll()`（または同等の関数）が**1つ**だけです。



ノンブロッキングのファイルディスクリプタを使用する必要があるため、`poll()`（または同等の関数）を使用せずに `read/recv` または `write/send` 関数を使用することも可能であり、その場合でもサーバーはブロッキング状態にはなりません。

ただし、その場合はより多くのシステムリソースを消費することになります。

したがって、`poll()`（または同等の関数）を使用せずにファイル記述子に対して `read/recv` または `write/send` を実行しようとする、評価は 0 となります。

- IRC クライアントはいくつか存在します。その中から 1 つをリファレンスとして選択してください。評価プロセスでは、このリファレンスクライアントが使用されます。
- 参照クライアントは、エラーが発生することなくサーバーに接続できるものでなければなりません。
- クライアントとサーバー間の通信は、TCP/IP（v4 または v6）を介して行わなければなりません。
- 参照クライアントをサーバーで使用した際の動作は、公式の IRC サーバーで使用した場合と同様でなければなりません。ただし、実装する必要がある機能は以下のものに限られます：
  - 参照クライアントを使用して、認証、ニックネームやユーザー名の設定、チャンネルへの参加、プライベートメッセージの送受信が可能でなければなりません。
  - あるクライアントからチャンネルに送信されたすべてのメッセージは、そのチャンネルに参加している他のすべてのクライアントに転送されなければなりません。
  - オペレーターと一般ユーザーが存在する必要があります。
  - さらに、チャンネルオペレーター専用のコマンドを実装する必要があります：
    - \* KICK - クライアントをチャンネルから追放する
    - \* INVITE - クライアントをチャンネルに招待する
    - \* TOPIC - チャンネルのトピックを変更または表示する
    - \* MODE - チャンネルのモードを変更する：
      - i: 招待制チャンネルの設定／解除
      - t: チャンネルオペレーターに対する TOPIC コマンドの制限を設定／解除する
      - k: チャンネルのキー（パスワード）を設定／解除する

- o: チャンネルオペレーター権限の付与・解除
  - l: チャンネルへのユーザー制限を設定／解除する
- もちろん、クリーンなコードを書くことが求められます。

## IV.2 MacOS のみ



MacOS では、他の Unix OS とは異なる方法で `write()` が実装されているため、`fcntl()` の使用が許可されています。

他の Unix OS と同様の動作を得るためには、ファイルディスクリプタをノンブロッキングモードで使用する必要があります。



ただし、`fcntl()` は次のようにのみ使用できます : `fcntl(fd, F_SETFL, O_NONBLOCK);`  
その他のフラグの使用は禁止されています。

## IV.3 テスト例

データの不完全な受信や帯域幅の不足など、考えられるあらゆるエラーや問題を確認してください。

サーバーが送信されたすべてのデータを正しく処理していることを確認するために、`nc` を使用した以下の簡単なテストを実行できます。

```
\$> nc -C 127.0.0.1 6667
com^Dman^Dd
\$>
```

`Ctrl+D` を使用して、コマンドを「com」、「man」、「d\n」の順に分割して送信します。

コマンドを処理するには、まず受信したパケットを統合してコマンドを再構築する必要があります。

## 第 V 章

### READMEの要件

Gitリポジトリのルートディレクトリに、README.md ファイルを必ず配置してください。このファイルの目的は、プロジェクトに不慣れな人（同僚、スタッフ、採用担当者など）が、プロジェクトの内容、実行方法、および関連情報の入手先を素早く理解できるようにすることです。

README.md には、少なくとも以下の内容を含める必要があります。

- 最初の行はイタリック体で、次のように記述してください：このプロジェクトは、`<login1>[、<login2>[、<login3>[...]]]` による 42 カリキュラムの一環として作成されました。
- プロジェクトの目的や概要を明確に説明した「Description」セクション。
- コンパイル、インストール、および／または実行に関する関連情報を記載した「Instructions」セクション。
- 「Resources」セクション：トピックに関連する主要な参考資料（ドキュメント、記事、チュートリアルなど）を列挙するとともに、AIがどのように使用されたか（どのタスクで、プロジェクトのどの部分で使用されたかを具体的に明記）を記述すること。

➡ プロジェクトによっては、追加のセクション（使用例、機能一覧、技術的な選択など）が必要になる場合があります。

必要な追加事項は、以下に明示的に記載されます。



README は英語で記述してください。

## 第 VI 章 ボーナスパ

### ート

実際の IRC サーバーにより近づけるために、IRC サーバーに追加できる機能は以下の通りです。

- ファイル転送機能を実装する。
- ボット。



ボーナス課題は、必須課題が「完璧」である場合にのみ評価されます。「完璧」とは、必須課題が完全に実装され、不具合なく動作していることを意味します。必須要件をすべて満たしていない場合、ボーナス課題は一切評価されません。

## 第VII章

### 提出とピア評価

課題は通常通り、Gitリポジトリに提出してください。発表会では、リポジトリ内の作業内容のみが評価の対象となります。ファイル名が正しいかどうか、念のため再確認してください。

**提出や採点の対象にはなりません**が、プロジェクト用のテストプログラムを作成することを推奨します。これらのテストは、発表時に自身のサーバーをテストするのに特に役立つだけでなく、将来他の `fitirc` を評価する際に、その人のサーバーをテストするのに役立ちます。実際、評価プロセス中は必要なテストを自由に使用して構いません。



評価プロセスでは、リファレンスクライアントが使用されます。

評価の過程で、プロジェクトへの軽微な**修正**が求められる場合があります。これには、動作のわずかな変更、数行のコードの記述や書き換え、あるいは簡単に追加できる機能の追加などが含まれる可能性があります。

このステップは**すべてのプロジェクトに当てはまるわけではありません**が、評価ガイドラインに記載されている場合は、その準備をしておく必要があります。

このステップは、プロジェクトの特定の部分について、あなたが実際に理解しているかどうかを確認するためのものです。変更は、任意の開発環境（例：普段使用している環境）で実行でき、評価の一環として特定の時間枠が定められていない限り、数分以内に完了できるものでなければなりません。例えば、関数やスクリプトの小さな更新、表示の変更、新しい情報を格納するためのデータ構造の調整などが求められる場合があります。

詳細（範囲、対象など）は**評価ガイドライン**に明記されており、同じプロジェクトであっても評価ごとに異なる場合があります。